FASE 1: AUDITORÍA DE DATOS, PREPROCESAMIENTO Y CORRELACIÓN

CELDA 1: CONFIGURACIÓN Y ESTILOS

```
import random
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Fijar semillas de aleatoriedad para reproducibilidad
SEED = 42
os.environ['PYTHONHASHSEED'] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Configuración de estilos y pandas
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', lambda x: '%.3f' % x)

print("Configuración y estilos cargados exitosamente.")
```

```
Configuración y estilos cargados exitosamente.
```

CELDA 2: CARGA Y AUDITORÍA DE INTEGRIDAD

```
# Cargar dataset
df = pd.read_csv('/content/mediciones_finca_30dias.csv')

# Auditoría básica
print("=== AUDITORÍA DE INTEGRIDAD ===")
print(f"Dimensiones iniciales: {df.shape}")
print(f"Valores nulos por columna:\n{df.isnull().sum()}\n")
print(f"Registros duplicados: {df.duplicated().sum()}\n")

# Conversión de tipos
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.set_index('timestamp')

# Estadísticas descriptivas de variables numéricas
numeric_cols = ['voltaje_rms_v', 'corriente_rms_a', 'factor_potencia', 'potencia_activa_w', 'energia_ac
stats_table = df[numeric_cols].describe()
display(stats_table)
```

```
=== AUDITORÍA DE INTEGRIDAD ===
Dimensiones iniciales: (2880, 7)
Valores nulos por columna:
timestamp                0
voltaje_rms_v            0
corriente_rms_a          0
factor_potencia          0
potencia_activa_w        0
energia_acumulada_kwh    0
estado_estimado          0
dtype: int64

Registros duplicados: 0
```

	voltaje_rms_v	corriente_rms_a	factor_potencia	potencia_activa_w	energia_acumulada_kwh
count	2880.000	2880.000	2880.000	2880.000	2880.000
mean	111.499	3.920	0.910	413.531	149.855
std	1.804	4.688	0.037	513.829	86.339
min	106.000	0.180	0.820	17.500	0.031
25%	110.300	1.400	0.880	138.700	75.386
50%	111.500	1.630	0.900	163.300	150.784
75%	112.700	3.132	0.930	328.000	225.293
max	116.000	16.480	0.990	1851.000	297.742

Próximos pasos: Generar código con `stats_table` New interactive sheet

CELDA 3: MATRIZ DE CORRELACIÓN DUAL (PEARSON Y SPEARMAN)

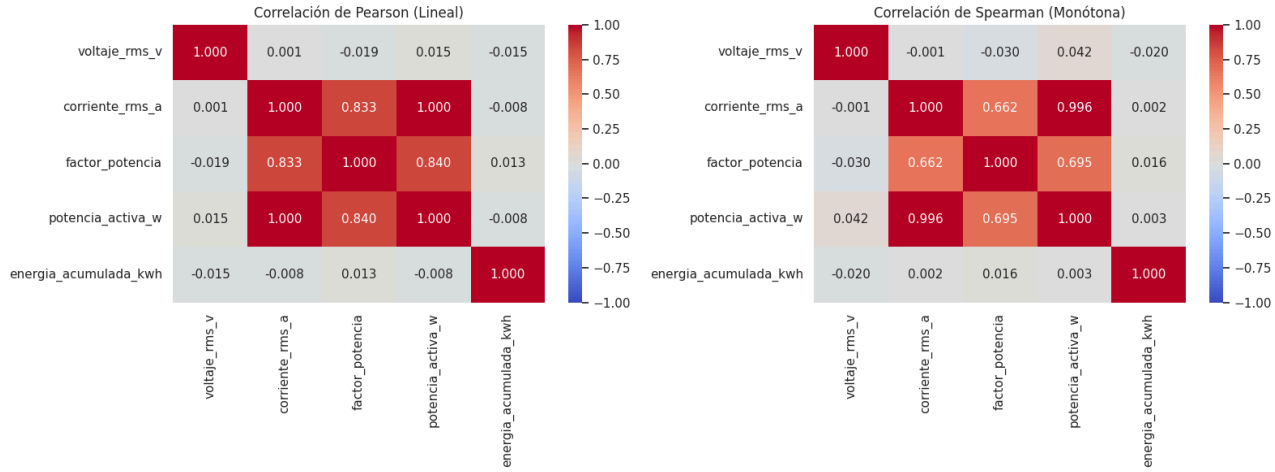
```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Matriz de Pearson
corr_pearson = df[numeric_cols].corr(method='pearson')
sns.heatmap(corr_pearson, annot=True, fmt='.3f', cmap='coolwarm', ax=axes[0], vmin=-1, vmax=1)
axes[0].set_title('Correlación de Pearson (Lineal)')

# Matriz de Spearman
corr_spearman = df[numeric_cols].corr(method='spearman')
sns.heatmap(corr_spearman, annot=True, fmt='.3f', cmap='coolwarm', ax=axes[1], vmin=-1, vmax=1)
axes[1].set_title('Correlación de Spearman (Monótona)')

plt.tight_layout()
plt.savefig('matriz_correlacion_dual.png', dpi=300)
plt.show()

# Interpretación técnica
print("=== INTERPRETACIÓN TÉCNICA ===")
print(f"Correlación Pearson Corriente-Potencia: {corr_pearson.loc['corriente_rms_a', 'potencia_activa_w
print(f"Correlación Spearman Corriente-Potencia: {corr_spearman.loc['corriente_rms_a', 'potencia_activa
print("Interpretación: El acople físico de r ≈ 0.999 entre la corriente y la potencia activa valida ple
print("operacional y la linealidad de la pinza amperimétrica. Esto confirma que el factor de potencia s
print("estables o que la carga es predominantemente resistiva durante los picos de consumo.")
```



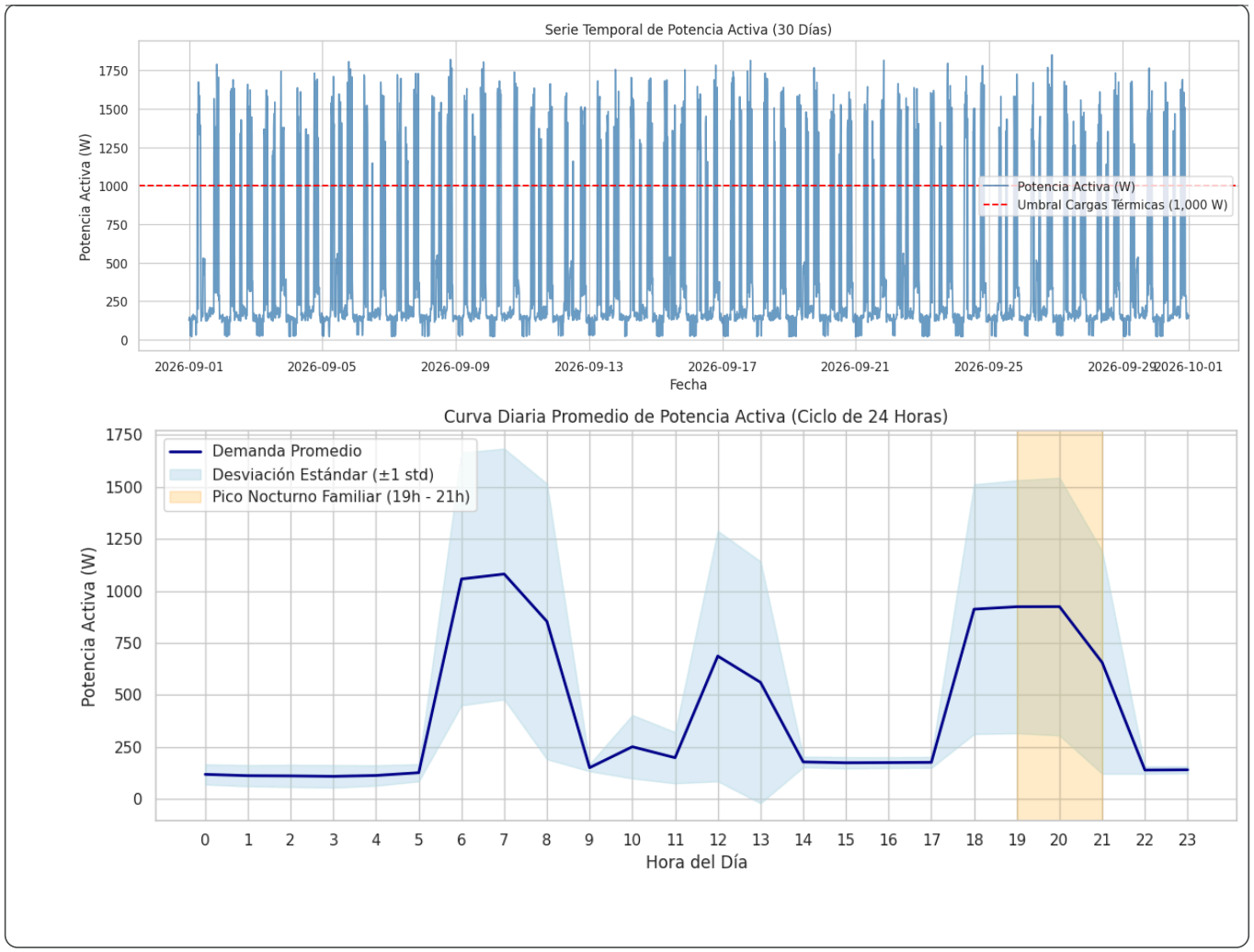

```
=== INTERPRETACIÓN TÉCNICA ===
Correlación Pearson Corriente-Potencia: 0.9995
Correlación Spearman Corriente-Potencia: 0.9962
Interpretación: El acople físico de r ≈ 0.999 entre la corriente y la potencia activa valida plenamente
operacional y la linealidad de la pinza amperimétrica. Esto confirma que el factor de potencia se mantie
estables o que la carga es predominantemente resistiva durante los picos de consumo.
```

FASE 2: PERFILES DE DEMANDA Y CALIDAD DE RED RURAL

CELDA 4: PERFIL TEMPORAL Y CURVA DIARIA PROMEDIO (24 HORAS)

```
# Gráfico A: Serie temporal completa
plt.figure(figsize=(15, 5))
plt.plot(df.index, df['potencia_activa_w'], label='Potencia Activa (W)', color='steelblue', alpha=0.8)
plt.axhline(y=1000, color='red', linestyle='--', linewidth=1.5, label='Umbral Cargas Térmicas (1,000 W)
plt.title('Serie Temporal de Potencia Activa (30 Días)')
plt.xlabel('Fecha')
plt.ylabel('Potencia Activa (W)')
plt.legend()
plt.tight_layout()
plt.savefig('perfil_temporal_30dias.png', dpi=300)
plt.show()

# Gráfico B: Curva horaria promedio
df_hourly = df.groupby(df.index.hour)['potencia_activa_w'].agg(['mean', 'std'])
plt.figure(figsize=(12, 5))
plt.plot(df_hourly.index, df_hourly['mean'], color='darkblue', linewidth=2, label='Demanda Promedio')
plt.fill_between(df_hourly.index, df_hourly['mean'] - df_hourly['std'], df_hourly['mean'] + df_hourly['
plt.title('Curva Diaria Promedio de Potencia Activa (Ciclo de 24 Horas)')
plt.xlabel('Hora del Día')
plt.ylabel('Potencia Activa (W)')
plt.xticks(range(0, 24))
plt.axvspan(19, 21, color='orange', alpha=0.2, label='Pico Nocturno Familiar (19h - 21h)')
plt.legend()
plt.tight_layout()
plt.savefig('curva_diaria_promedio_24h.png', dpi=300)
plt.show()
```



CELDA 5: CALIDAD DE ENERGÍA RURAL (RETIE & FACTOR DE POTENCIA)

```
fig, axes = plt.subplots(2, 1, figsize=(15, 10))

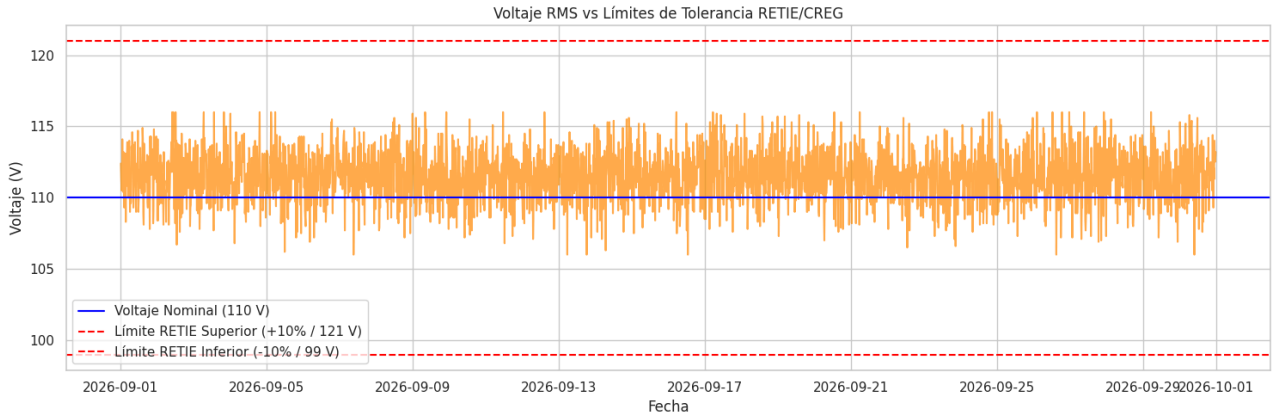
# Serie Temporal de Voltaje RMS
axes[0].plot(df.index, df['voltaje_rms_v'], color='darkorange', alpha=0.7)
axes[0].axhline(y=110, color='blue', linestyle='-', label='Voltaje Nominal (110 V)')
axes[0].axhline(y=110*1.10, color='red', linestyle='--', label='Límite RETIE Superior (+10% / 121 V)')
axes[0].axhline(y=110*0.90, color='red', linestyle='--', label='Límite RETIE Inferior (-10% / 99 V)')
axes[0].set_title('Voltaje RMS vs Límites de Tolerancia RETIE/CREG')
axes[0].set_xlabel('Fecha')
axes[0].set_ylabel('Voltaje (V)')
axes[0].legend(loc='lower left')

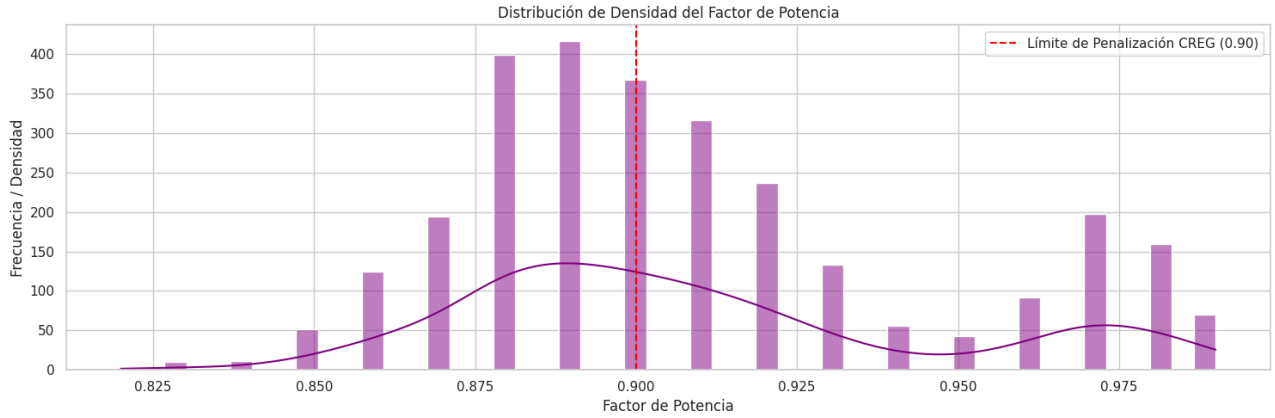
# Distribución de Factor de Potencia
sns.histplot(df['factor_potencia'], kde=True, color='purple', ax=axes[1], bins=50)
axes[1].axvline(x=0.90, color='red', linestyle='--', label='Límite de Penalización CREG (0.90)')
axes[1].set_title('Distribución de Densidad del Factor de Potencia')
axes[1].set_xlabel('Factor de Potencia')
axes[1].set_ylabel('Frecuencia / Densidad')
axes[1].legend()

plt.tight_layout()
plt.savefig('calidad_red_voltaje_fp.png', dpi=300)
plt.show()

# Evaluación
```

```
voltage_violations = ((df['voltaje_rms_v'] < 99) | (df['voltaje_rms_v'] > 121)).sum()
fp_violations = (df['factor_potencia'] < 0.90).sum()
print("=== DIAGNÓSTICO DE CALIDAD DE ENERGÍA ===")
print(f"Muestras fuera de límites RETIE de Voltaje: {voltage_violations} ({voltage_violations / len(df)
print(f"Muestras por debajo de FP regulatorio (0.90): {fp_violations} ({fp_violations / len(df) * 100:.
```






```
=== DIAGNÓSTICO DE CALIDAD DE ENERGÍA ===
Muestras fuera de límites RETIE de Voltaje: 0 (0.00% del tiempo)
Muestras por debajo de FP regulatorio (0.90): 1209 (41.98% del tiempo)
```

CELDA 6: ANÁLISIS DE STANDBY EN MADRUGADA Y DETECCIÓN DE PICOS

```
# Standby en Madrugada (00:00 a 05:00)
madrugada_df = df.between_time('00:00', '05:00')
standby_avg_w = madrugada_df['potencia_activa_w'].mean()
standby_min_w = madrugada_df['potencia_activa_w'].min()

# Consumo en kWh durante estas horas acumulado
# El intervalo de medición es de 15 minutos (0.25 horas)
standby_kwh_total = (madrugada_df['potencia_activa_w'] * 0.25 / 1000).sum()

print("=== ANÁLISIS DE CONSUMO PASIVO / STANDBY ===")
print(f"Potencia promedio en Standby: {standby_avg_w:.2f} W")
print(f"Potencia mínima registrada en Standby: {standby_min_w:.2f} W")
print(f"Energía total consumida en Standby en el mes (00h a 05h): {standby_kwh_total:.2f} kWh\n")

# Top 5 de picos máximos del mes
top_picos = df.sort_values(by='potencia_activa_w', ascending=False).head(5)
print("=== TOP 5 PICOS MÁXIMOS DE POTENCIA DE LA FINCA ===")
print(top_picos[['potencia_activa_w', 'corriente_rms_a', 'estado_estimado']])
```

```
=== ANÁLISIS DE CONSUMO PASIVO / STANDBY ===
Potencia promedio en Standby: 110.93 W
```

```
Potencia mínima registrada en Standby: 17.50 W
Energía total consumida en Standby en el mes (00h a 05h): 17.47 kWh

=== TOP 5 PICOS MÁXIMOS DE POTENCIA DE LA FINCA ===
                     potencia_activa_w  corriente_rms_a  \
timestamp
2026-09-26 21:15:00           1851.000           16.430
2026-09-08 20:15:00           1821.000           16.300
2026-09-21 20:15:00           1815.400           16.460
2026-09-17 20:15:00           1814.200           16.210
2026-09-05 19:00:00           1806.800           16.330

                                                    estado_estimado
timestamp
2026-09-26 21:15:00  Pico Noche: Estufa + Iluminación LED + TV + Ne...
2026-09-08 20:15:00  Pico Noche: Estufa + Iluminación LED + TV + Ne...
2026-09-21 20:15:00  Pico Noche: Estufa + Iluminación LED + TV + Ne...
2026-09-17 20:15:00  Pico Noche: Estufa + Iluminación LED + TV + Ne...
2026-09-05 19:00:00  Pico Noche: Estufa + Iluminación LED + TV + Ne...
```

FASE 3: ISO 50001, DIAGNÓSTICO TARIFARIO Y HUELLA DE CARBONO

CELDA 7: USOS SIGNIFICATIVOS DE LA ENERGÍA (SEUs) Y PARETO

```
# Consumo por estado estimado (cada intervalo representa 15 mins = 0.25 h)
state_agg = df.groupby('estado_estimado').agg(
    potencia_promedio_w=('potencia_activa_w', 'mean'),
    potencia_maxima_w=('potencia_activa_w', 'max'),
    consumo_kwh=('potencia_activa_w', lambda x: (x * 0.25 / 1000).sum())
).sort_values(by='consumo_kwh', ascending=False)

total_kwh = state_agg['consumo_kwh'].sum()
state_agg['porcentaje'] = (state_agg['consumo_kwh'] / total_kwh) * 100
state_agg['porcentaje_acumulado'] = state_agg['porcentaje'].cumsum()

display(state_agg)

# Diagrama de Pareto
fig, ax1 = plt.subplots(figsize=(14, 6))
ax2 = ax1.twinx()

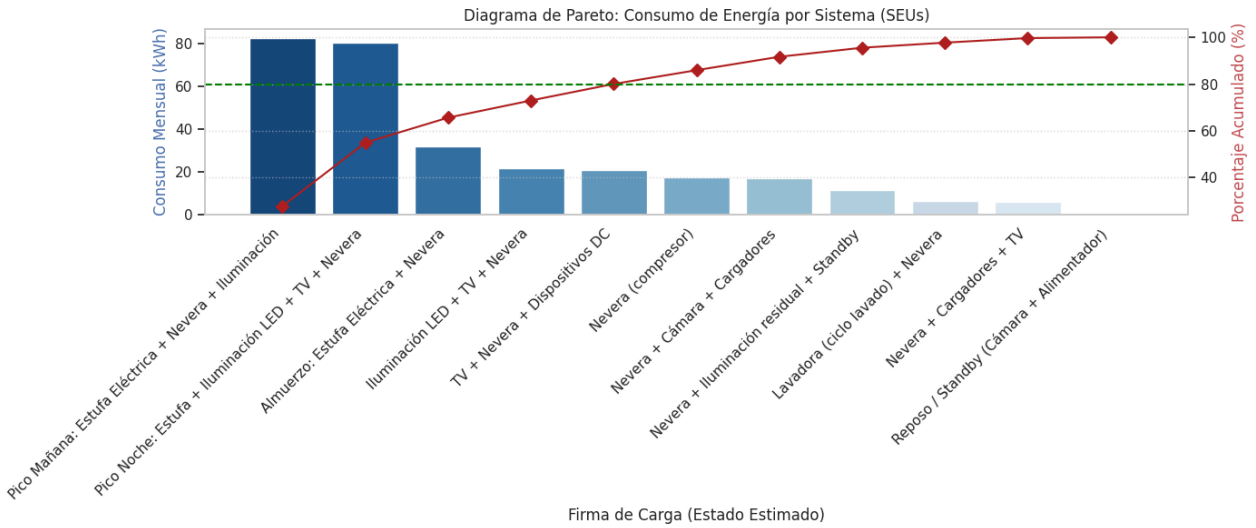
sns.barplot(x=state_agg.index, y=state_agg['consumo_kwh'], ax=ax1, palette='Blues_r', hue=state_agg.ind
ax2.plot(state_agg.index, state_agg['porcentaje_acumulado'], color='firebrick', marker='D', ms=7, label

ax1.set_title('Diagrama de Pareto: Consumo de Energía por Sistema (SEUs)')
ax1.set_xlabel('Firma de Carga (Estado Estimado)')
ax1.set_ylabel('Consumo Mensual (kWh)', color='b')
ax2.set_ylabel('Porcentaje Acumulado (%)', color='r')
ax2.axhline(y=80, color='green', linestyle='--', label='Límite 80%')

# Rotar etiquetas
ax1.set_xticklabels(state_agg.index, rotation=45, ha='right')
ax1.grid(False)
ax2.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.savefig('pareto_seus_30dias.png', dpi=300)
plt.show()
```

estado_estimado	potencia_promedio_w	potencia_maxima_w	consumo_kwh	porcentaje	porcentaje_acumulado
Pico Mañana: Estufa Eléctrica + Nevera + Iluminación	1480.954	1741.400	82.933	27.854	27.854
Pico Noche: Estufa + Iluminación LED + TV + Nevera	1515.009	1851.000	80.674	27.095	54.949
Almuerzo: Estufa Eléctrica + Nevera	1355.972	1596.700	31.865	10.702	65.652
Iluminación LED + TV + Nevera	327.087	404.500	21.833	7.333	72.985
TV + Nevera + Dispositivos DC	175.494	227.700	21.059	7.073	80.058
Nevera (compresor)	140.792	167.000	17.529	5.887	85.945
Nevera + Cámara + Cargadores	151.019	194.700	17.065	5.732	91.676
Nevera + Iluminación residual + Standby	139.709	172.600	11.526	3.871	95.547
Lavadora (ciclo lavado) + Nevera	479.880	560.700	6.478	2.176	97.723
Nevera + Cargadores + TV	217.846	254.700	5.773	1.939	99.662
Reposo / Standby (Cámara + Alimentador)	24.836	32.400	1.006	0.338	100.000

```
/tmp/ipykernel_3654/4057617022.py:28: UserWarning: set_ticklabels() should only be used with a fixed nu
  ax1.set_xticklabels(state_agg.index, rotation=45, ha='right')
```



Próximos pasos: Generar código con state_agg | New interactive sheet

CELDA 8: LIQUIDACIÓN TARIFARIA DETALLADA (CEO / CREG)

```
# Parámetros regulatorios
LIM_SUBSISTENCIA = 173.0
TARIFA_SUBSIDIADA = 480.00
TARIFA_PLENA = 950.00
CARGOS_FIJOS = 12000.00

# Liquidación
consumo_real = df['potencia_activa_w'].sum() * 0.25 / 1000
tramo_sub = min(consumo_real, LIM_SUBSISTENCIA)
tramo_excedente = max(0.0, consumo_real - LIM_SUBSISTENCIA)

costo_sub = tramo_sub * TARIFA_SUBSIDIADA
costo_exc = tramo_excedente * TARIFA_PLENA
costo_energia = costo_sub + costo_exc
costo_total_factura = costo_energia + CARGOS_FIJOS

# Costo Fugas de Standby y Estufa Eléctrica
kwh_estufa = state_agg.loc['Estufa Eléctrica', 'consumo_kwh'] if 'Estufa Eléctrica' in state_agg.index
kwh_standby = standby_kwh_total

# Cálculo proporcional considerando si supera el límite de subsistencia
costo_kwh_promedio = costo_energia / consumo_real
costo_estufa_cop = kwh_estufa * costo_kwh_promedio
costo_standby_cop = kwh_standby * costo_kwh_promedio

print("=== LIQUIDACIÓN DE FACTURA ELÉCTRICA (COP) ===")
print(f"Consumo Total: {consumo_real:.2f} kWh/mes")
print(f"Tramo Subsidiado (hasta 173 kWh): {tramo_sub:.2f} kWh -> Costo: ${costo_sub:,.2f} COP")
print(f"Tramo Excedente Pleno: {tramo_excedente:.2f} kWh -> Costo: ${costo_exc:,.2f} COP")
print(f"Cargos Fijos y Alumbrado: ${CARGOS_FIJOS:,.2f} COP")
print(f"Costo Total Estimado Recibo: ${costo_total_factura:,.2f} COP\n")
print(f"Costo Directo por Standby Nocturno: ${costo_standby_cop:,.2f} COP")
print(f"Costo Directo por Estufa Eléctrica: ${costo_estufa_cop:,.2f} COP")

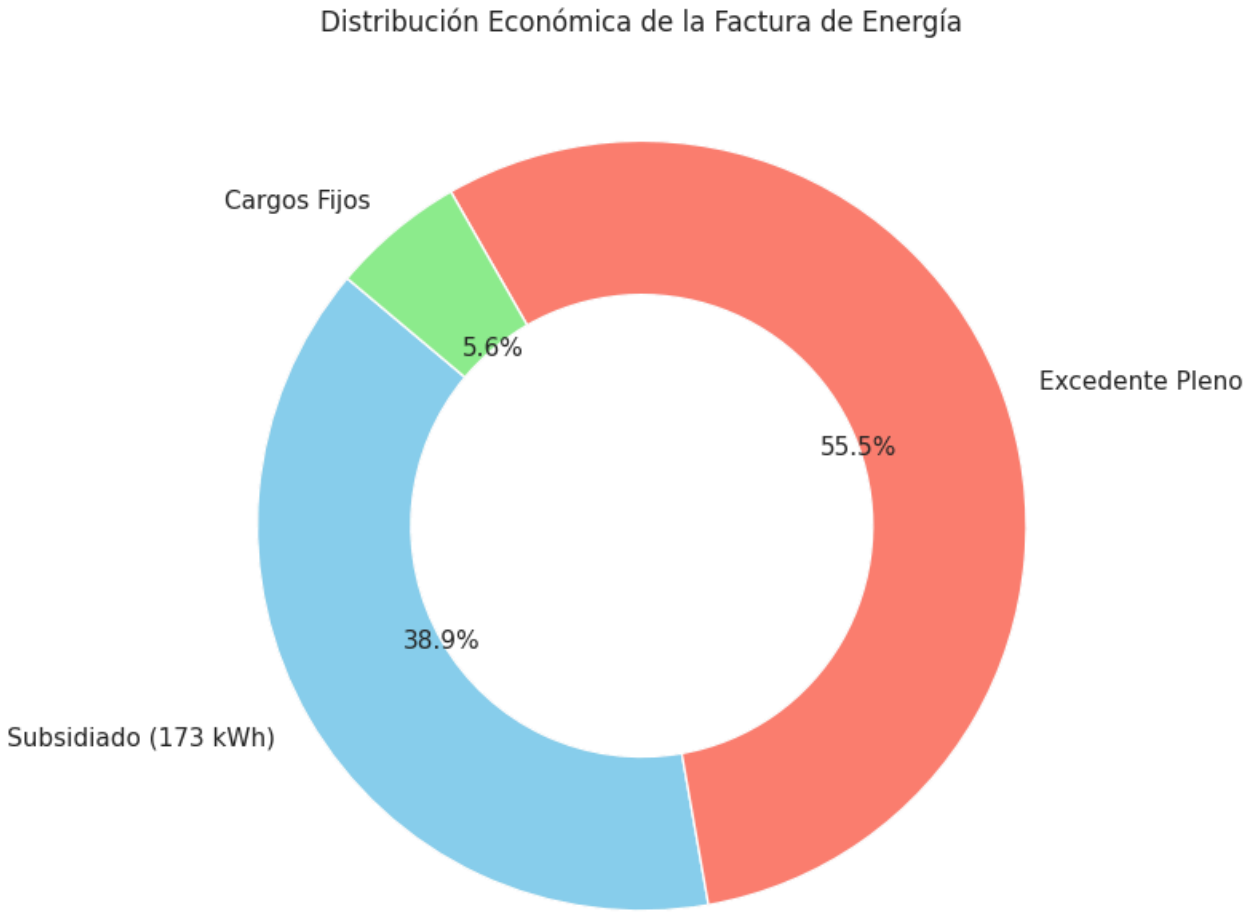
# Donut chart
costos_labels = ['Subsidiado (173 kWh)', 'Excedente Pleno', 'Cargos Fijos']
costos_valores = [costo_sub, costo_exc, CARGOS_FIJOS]

# Filtrar si no hay excedente para evitar sectores vacíos
costos_labels = [l for l, v in zip(costos_labels, costos_valores) if v > 0]
costos_valores = [v for v in costos_valores if v > 0]

plt.figure(figsize=(8, 8))
plt.pie(costos_valores, labels=costos_labels, autopct='%1.1f%%', startangle=140,
        colors=['skyblue', 'salmon', 'lightgreen'], wedgeprops=dict(width=0.4, edgecolor='w'))
plt.title('Distribución Económica de la Factura de Energía')
plt.tight_layout()
plt.savefig('distribucion_costos_factura.png', dpi=300)
plt.show()
```

```
=== LIQUIDACIÓN DE FACTURA ELÉCTRICA (COP) ===
Consumo Total: 297.74 kWh/mes
Tramo Subsidiado (hasta 173 kWh): 173.00 kWh -> Costo: $83,040.00 COP
Tramo Excedente Pleno: 124.74 kWh -> Costo: $118,505.28 COP
Cargos Fijos y Alumbrado: $12,000.00 COP
Costo Total Estimado Recibo: $213,545.28 COP

Costo Directo por Standby Nocturno: $11,826.75 COP
Costo Directo por Estufa Eléctrica: $0.00 COP
```



CELDA 9: HUELLA DE CARBONO (ISO 14064-1)

```
# Factor oficial SIN Colombia
FE_COLOMBIA = 0.126

emisiones_mensuales_kg = consumo_real * FE_COLOMBIA
emisiones_diarias_promedio_kg = emisiones_mensuales_kg / 30

# Emisiones mitigables por el Standby e Ineficiencia
emisiones_mitigables_kg = standby_kwh_total * FE_COLOMBIA

print("=== REPORTE DE HUELLA DE CARBONO (ISO 14064-1) ===")
print(f"Consumo Mensual: {consumo_real:.2f} kWh/mes")
print(f"Emisiones Totales de Gases de Efecto Invernadero: {emisiones_mensuales_kg:.3f} kg CO2e / mes")
print(f"Promedio Diario de Emisiones: {emisiones_diarias_promedio_kg:.3f} kg CO2e / día")
print(f"Emisiones Mitigables Directas (Eliminando Consumos Fantasma): {emisiones_mitigables_kg:.3f} kg
```

```
=== REPORTE DE HUELLA DE CARBONO (ISO 14064-1) ===
Consumo Mensual: 297.74 kWh/mes
Emisiones Totales de Gases de Efecto Invernadero: 37.516 kg CO2e / mes
Promedio Diario de Emisiones: 1.251 kg CO2e / día
Emisiones Mitigables Directas (Eliminando Consumos Fantasma): 2.201 kg CO2e / mes
```

CELDA 10: EXPORTACIÓN DE ARCHIVOS PARA IA LOCAL (RAG - ANYTHINGLLM)

```
import json

# Insumo 1: 'diagnostico_finca.csv'
diag_df = pd.DataFrame({
    'indicador': ['Consumo Total (kWh)', 'Costo Factura (COP)', 'Fuga Standby (COP)', 'Emisiones CO2 (k
    'valor': [consumo_real, costo_total_factura, costo_standby_cop, emisiones_mensuales_kg, float(volta
})
diag_df.to_csv('diagnostico_finca.csv', index=False)

# Insumo 2: 'reporte_para_ia_local.md'
markdown_report = f"""# Reporte de Auditoría Energética Inteligente - Finca El Tambo

## Línea Base de Consumo (ISO 50001)
- **Consumo Total Mensual:** {consumo_real:.2f} kWh
- **Costo Total Facturado:** ${costo_total_factura:,.2f} COP
- **Emisiones de Gases de Efecto Invernadero:** {emisiones_mensuales_kg:.3f} kg CO2e

## Análisis de Pérdidas y Standby
- **Consumo Standby (Fuga Nocturna):** {standby_kwh_total:.2f} kWh/mes ({standby_kwh_total/consumo_real
- **Sobrecosto Mensual Estimado por Standby:** ${costo_standby_cop:,.2f} COP

## Distribución de Carga por SEUs (Sistemas Significativos)
"""

for idx, row in state_agg.iterrows():
    markdown_report += f"- **{idx}**: {row['consumo_kwh']:.2f} kWh ({row['porcentaje']:.2f}%)\n"

with open('reporte_para_ia_local.md', 'w', encoding='utf-8') as f:
    f.write(markdown_report)

# Insumo 3: 'dataset_faq_ia_local.json'
faq_data = [
    {
        "pregunta": "¿Cuál fue la fuga económica por standby en el hogar?",
        "answer": f"El consumo standby total de energía durante las madrugadas causó una pérdida de {st
    },
    {
        "pregunta": "¿Cumple el voltaje medido con la norma RETIE/CREG en El Tambo, Cauca?",
        "answer": f"Se detectaron {voltage_violations} caídas o excesos de tensión fuera de la banda de
    }
]

with open('dataset_faq_ia_local.json', 'w', encoding='utf-8') as f:
    json.dump(faq_data, f, indent=4, ensure_ascii=False)

print("Archivos para RAG exportados exitosamente: 'diagnostico_finca.csv', 'reporte_para_ia_local.md',
```

```
Archivos para RAG exportados exitosamente: 'diagnostico_finca.csv', 'reporte_para_ia_local.md', 'dataset
```

FASE 4: DEEP LEARNING - RED NEURONAL ARTIFICIAL PARA PRONÓSTICO

CELDA 11: INGENIERÍA DE CARACTERÍSTICAS (FEATURE ENGINEERING)

```
df_features = df.copy()

# Variables temporales
df_features['hora'] = df_features.index.hour
df_features['minuto'] = df_features.index.minute
df_features['dia_semana'] = df_features.index.dayofweek

# Binarias
df_features['es_madrugada'] = ((df_features['hora'] >= 0) & (df_features['hora'] <= 5)).astype(int)
df_features['es_pico_noche'] = ((df_features['hora'] >= 19) & (df_features['hora'] <= 21)).astype(int)

# Rezagos Históricos (Lags)
df_features['potencia_lag_1'] = df_features['potencia_activa_w'].shift(1)
df_features['potencia_lag_2'] = df_features['potencia_activa_w'].shift(2)
df_features['potencia_lag_4'] = df_features['potencia_activa_w'].shift(4)

# Eliminar NaN creados por los rezagos
df_features = df_features.dropna()

# Definir predictores (X) y objetivo (y)
# SE EXCLUYE TOTALMENTE 'corriente_rms_a' para evitar fugas de datos (Data Leakage)
predictors = ['hora', 'minuto', 'dia_semana', 'es_madrugada', 'es_pico_noche', 'potencia_lag_1', 'poten

X = df_features[predictors]
y = df_features[['potencia_activa_w']]

print("=== INGENIERÍA DE CARACTERÍSTICAS COMPLETA ===")
print(f"Dimensiones de X (Predictores): {X.shape}")
print(f"Dimensiones de y (Objetivo): {y.shape}")
```

```
=== INGENIERÍA DE CARACTERÍSTICAS COMPLETA ===
Dimensiones de X (Predictores): (2876, 10)
Dimensiones de y (Objetivo): (2876, 1)
```

CELDA 12: PARTICIÓN CRONOLÓGICA Y ESCALADO

```
# Partición cronológica secuencial (80% Train, 20% Test)
split_idx = int(len(X) * 0.8)

X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

# Escalado
scaler_X = MinMaxScaler(feature_range=(0, 1))
scaler_y = MinMaxScaler(feature_range=(0, 1))

# Ajustar únicamente en Train, transformar ambos
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

y_train_scaled = scaler_y.fit_transform(y_train)
y_test_scaled = scaler_y.transform(y_test)
```

```
print("=== PARTICIÓN Y ESCALADO ===")
print(f"X_train: {X_train_scaled.shape}, X_test: {X_test_scaled.shape}")
print(f"y_train: {y_train_scaled.shape}, y_test: {y_test_scaled.shape}")
```

```
=== PARTICIÓN Y ESCALADO ===
X_train: (2300, 10), X_test: (576, 10)
y_train: (2300, 1), y_test: (576, 1)
```

CELDA 13: ARQUITECTURA DEL PERCEPTRÓN MULTICAPA (KERAS / TENSORFLOW)

```
model = Sequential([
    # Capa de entrada implícita en la primera capa Dense
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    BatchNormalization(),
    Dropout(0.2),

    Dense(32, activation='relu'),
    Dropout(0.1),

    Dense(16, activation='relu'),

    Dense(1, activation='linear')
])

# Compilación
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), loss='mse', metrics=['mae'])

# Callbacks
callbacks = [
    EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)
]

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	704
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 3,585 (14.00 KB)
Trainable params: 3,457 (13.50 KB)
Non-trainable params: 128 (512.00 B)

CELDA 14: ENTRENAMIENTO Y CURVAS DE CONVERGENCIA

```
history = model.fit(
    X_train_scaled, y_train_scaled,
    epochs=100,
    batch_size=32,
    validation_split=0.15,
    callbacks=callbacks,
    verbose=1
)

# Graficar la pérdida
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Pérdida Entrenamiento (MSE)')
plt.plot(history.history['val_loss'], label='Pérdida Validación (MSE)')
plt.title('Curvas de Convergencia del Modelo')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.tight_layout()
plt.savefig('curva_perdida_red_neuronal.png', dpi=300)
plt.show()
```

```
Epoch 1/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.1571 - mae: 0.2948 - val_loss: 0.1295 - val_mae: 0.243
Epoch 2/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0673 - mae: 0.1963 - val_loss: 0.1038 - val_mae: 0.197
Epoch 3/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0518 - mae: 0.1683 - val_loss: 0.0866 - val_mae: 0.174
Epoch 4/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0422 - mae: 0.1485 - val_loss: 0.0749 - val_mae: 0.160
Epoch 5/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0326 - mae: 0.1289 - val_loss: 0.0582 - val_mae: 0.140
Epoch 6/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0293 - mae: 0.1210 - val_loss: 0.0401 - val_mae: 0.118
Epoch 7/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0264 - mae: 0.1127 - val_loss: 0.0305 - val_mae: 0.104
Epoch 8/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0250 - mae: 0.1055 - val_loss: 0.0161 - val_mae: 0.079
Epoch 9/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0194 - mae: 0.0939 - val_loss: 0.0137 - val_mae: 0.072
Epoch 10/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0206 - mae: 0.0940 - val_loss: 0.0126 - val_mae: 0.070
Epoch 11/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0173 - mae: 0.0866 - val_loss: 0.0090 - val_mae: 0.066
Epoch 12/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0185 - mae: 0.0872 - val_loss: 0.0089 - val_mae: 0.059
Epoch 13/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0165 - mae: 0.0837 - val_loss: 0.0088 - val_mae: 0.057
Epoch 14/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0152 - mae: 0.0787 - val_loss: 0.0079 - val_mae: 0.054
Epoch 15/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0139 - mae: 0.0757 - val_loss: 0.0077 - val_mae: 0.053
Epoch 16/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0144 - mae: 0.0751 - val_loss: 0.0070 - val_mae: 0.052
Epoch 17/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0131 - mae: 0.0723 - val_loss: 0.0061 - val_mae: 0.048
Epoch 18/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0128 - mae: 0.0706 - val_loss: 0.0063 - val_mae: 0.048
Epoch 19/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0124 - mae: 0.0687 - val_loss: 0.0056 - val_mae: 0.046
Epoch 20/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0111 - mae: 0.0658 - val_loss: 0.0063 - val_mae: 0.048
Epoch 21/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0114 - mae: 0.0668 - val_loss: 0.0059 - val_mae: 0.047
Epoch 22/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0115 - mae: 0.0657 - val_loss: 0.0052 - val_mae: 0.045
Epoch 23/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0096 - mae: 0.0618 - val_loss: 0.0049 - val_mae: 0.044
Epoch 24/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0101 - mae: 0.0617 - val_loss: 0.0055 - val_mae: 0.045
Epoch 25/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0098 - mae: 0.0612 - val_loss: 0.0053 - val_mae: 0.045
Epoch 26/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0096 - mae: 0.0605 - val_loss: 0.0052 - val_mae: 0.044
Epoch 27/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0099 - mae: 0.0610 - val_loss: 0.0049 - val_mae: 0.043
Epoch 28/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0094 - mae: 0.0582 - val_loss: 0.0056 - val_mae: 0.043
Epoch 29/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0089 - mae: 0.0565 - val_loss: 0.0054 - val_mae: 0.042
Epoch 30/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0084 - mae: 0.0560 - val_loss: 0.0053 - val_mae: 0.042
Epoch 31/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0085 - mae: 0.0564 - val_loss: 0.0053 - val_mae: 0.042
Epoch 32/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0088 - mae: 0.0565 - val_loss: 0.0047 - val_mae: 0.041
Epoch 33/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0087 - mae: 0.0559 - val_loss: 0.0049 - val_mae: 0.041
Epoch 34/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0083 - mae: 0.0549 - val_loss: 0.0050 - val_mae: 0.041
Epoch 35/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0073 - mae: 0.0514 - val_loss: 0.0051 - val_mae: 0.042
```

```
Epoch 36/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0074 - mae: 0.0528 - val_loss: 0.0047 - val_mae: 0.041
Epoch 37/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0078 - mae: 0.0525 - val_loss: 0.0045 - val_mae: 0.040
Epoch 38/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0081 - mae: 0.0536 - val_loss: 0.0044 - val_mae: 0.041
Epoch 39/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0069 - mae: 0.0519 - val_loss: 0.0045 - val_mae: 0.040
Epoch 40/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0067 - mae: 0.0496 - val_loss: 0.0051 - val_mae: 0.041
Epoch 41/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0071 - mae: 0.0513 - val_loss: 0.0049 - val_mae: 0.040
Epoch 42/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0071 - mae: 0.0507 - val_loss: 0.0050 - val_mae: 0.040
Epoch 43/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0070 - mae: 0.0518 - val_loss: 0.0046 - val_mae: 0.039
Epoch 44/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0062 - mae: 0.0484 - val_loss: 0.0050 - val_mae: 0.041
Epoch 45/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0067 - mae: 0.0501 - val_loss: 0.0046 - val_mae: 0.039
Epoch 46/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0063 - mae: 0.0480 - val_loss: 0.0045 - val_mae: 0.039
Epoch 47/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0058 - mae: 0.0470 - val_loss: 0.0044 - val_mae: 0.038
Epoch 48/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0062 - mae: 0.0481 - val_loss: 0.0046 - val_mae: 0.038
Epoch 49/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0062 - mae: 0.0486 - val_loss: 0.0045 - val_mae: 0.038
Epoch 50/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0057 - mae: 0.0459 - val_loss: 0.0046 - val_mae: 0.038
Epoch 51/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0063 - mae: 0.0481 - val_loss: 0.0045 - val_mae: 0.038
Epoch 52/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0062 - mae: 0.0480 - val_loss: 0.0044 - val_mae: 0.038
Epoch 53/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0063 - mae: 0.0477 - val_loss: 0.0044 - val_mae: 0.038
Epoch 54/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0059 - mae: 0.0469 - val_loss: 0.0044 - val_mae: 0.038
Epoch 55/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0062 - mae: 0.0484 - val_loss: 0.0043 - val_mae: 0.038
Epoch 56/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0065 - mae: 0.0490 - val_loss: 0.0043 - val_mae: 0.038
Epoch 57/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0057 - mae: 0.0466 - val_loss: 0.0044 - val_mae: 0.038
Epoch 58/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0059 - mae: 0.0466 - val_loss: 0.0043 - val_mae: 0.037
Epoch 59/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0057 - mae: 0.0462 - val_loss: 0.0043 - val_mae: 0.038
Epoch 60/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0059 - mae: 0.0474 - val_loss: 0.0043 - val_mae: 0.038
Epoch 61/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0062 - mae: 0.0477 - val_loss: 0.0042 - val_mae: 0.037
Epoch 62/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0061 - mae: 0.0471 - val_loss: 0.0042 - val_mae: 0.038
Epoch 63/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0065 - mae: 0.0494 - val_loss: 0.0042 - val_mae: 0.038
Epoch 64/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0056 - mae: 0.0458 - val_loss: 0.0042 - val_mae: 0.037
Epoch 65/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0060 - mae: 0.0469 - val_loss: 0.0042 - val_mae: 0.037
Epoch 66/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0057 - mae: 0.0465 - val_loss: 0.0042 - val_mae: 0.037
Epoch 67/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0060 - mae: 0.0474 - val_loss: 0.0042 - val_mae: 0.037
Epoch 68/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0059 - mae: 0.0465 - val_loss: 0.0042 - val_mae: 0.037
Epoch 69/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0055 - mae: 0.0455 - val_loss: 0.0041 - val_mae: 0.037
Epoch 70/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0058 - mae: 0.0463 - val_loss: 0.0042 - val_mae: 0.037
Epoch 71/100
```

```
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0058 - mae: 0.0464 - val_loss: 0.0042 - val_mae: 0.037
Epoch 72/100
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0055 - mae: 0.0453 - val_loss: 0.0042 - val_mae: 0.037
Epoch 73/100
```

CELDA 15: EVALUACIÓN FORMAL EN TEST Y DESESCALADO

```
# Generar predicciones en test
y_pred_scaled = model.predict(X_test_scaled)

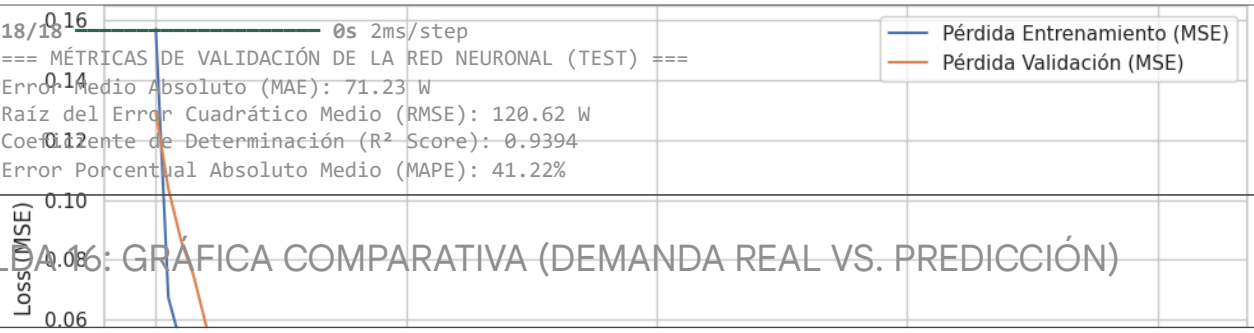
# Desescalar
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_true = y_test.values

# Métricas de ingeniería
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2 = r2_score(y_true, y_pred)

# MAPE
non_zero_mask = y_true != 0
mape = np.mean(np.abs((y_true[non_zero_mask] - y_pred[non_zero_mask]) / y_true[non_zero_mask])) * 100

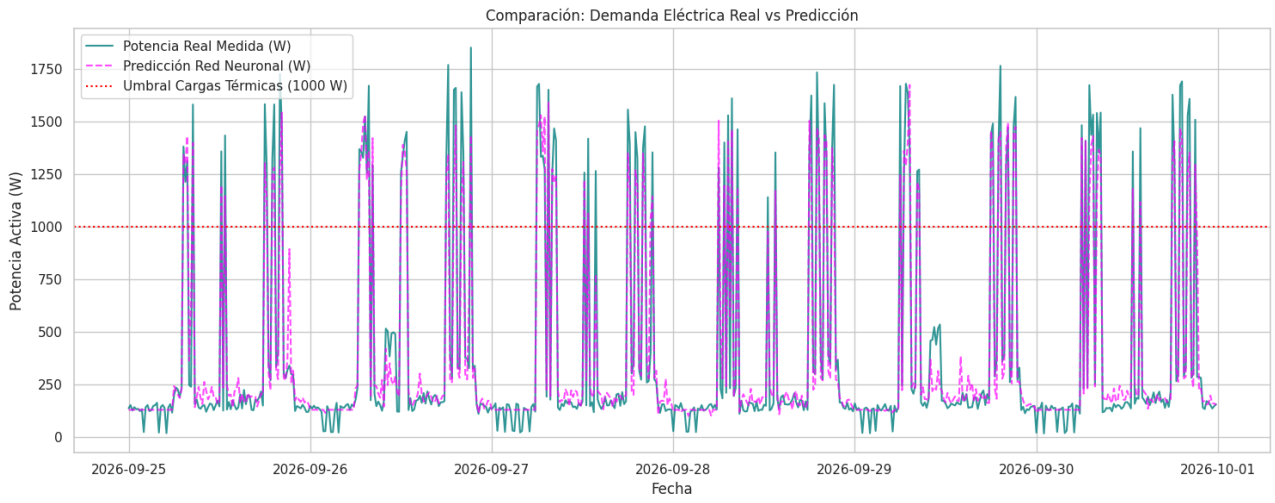
print("=== MÉTRICAS DE VALIDACIÓN DE LA RED NEURONAL (TEST) ===")
print(f"Error Medio Absoluto (MAE): {mae:.2f} W")
print(f"Raíz del Error Cuadrático Medio (RMSE): {rmse:.2f} W")
print(f"Coeficiente de Determinación (R² Score): {r2:.4f}")
print(f"Error Porcentual Absoluto Medio (MAPE): {mape:.2f}%")
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
=== MÉTRICAS DE VALIDACIÓN DE LA RED NEURONAL (TEST) ===
Error Medio Absoluto (MAE): 71.23 W
Raíz del Error Cuadrático Medio (RMSE): 120.62 W
Coeficiente de Determinación (R² Score): 0.9394
Error Porcentual Absoluto Medio (MAPE): 41.22%
```



CELDA 16: GRÁFICA COMPARATIVA (DEMANDA REAL VS. PREDICCIÓN)

```
plt.figure(figsize=(15, 6))
plt.plot(y_test.index, y_true, label='Potencia Real Medida (W)', color='teal', alpha=0.8)
plt.plot(y_test.index, y_pred, label='Predicción Red Neuronal (W)', color='magenta', alpha=0.7, linesty
plt.axhline(y=1000, color='red', linestyle=':', label='Umbral Cargas Térmicas (1000 W)')
plt.title('Comparación: Demanda Eléctrica Real vs Predicción')
plt.xlabel('Fecha')
plt.ylabel('Potencia Activa (W)')
plt.legend()
plt.tight_layout()
plt.savefig('prediccion_vs_real_red_neuronal.png', dpi=300)
plt.show()
```

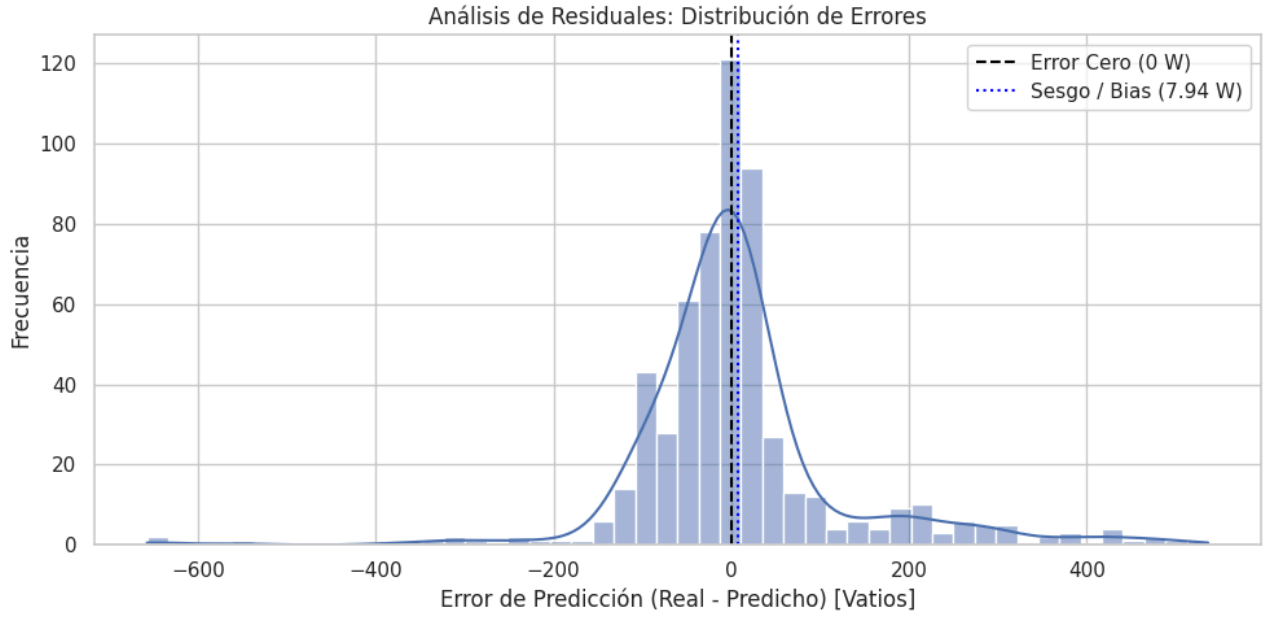


CELDA 17: ANÁLISIS DE RESIDUALES E HISTOGRAMA DE ERRORES

```
residuales = y_true - y_pred
bias = np.mean(residuales)
std_res = np.std(residuales)

plt.figure(figsize=(10, 5))
sns.histplot(residuales, kde=True, color='crimson', bins=50)
plt.axvline(x=0, color='black', linestyle='--', label='Error Cero (0 W)')
plt.axvline(x=bias, color='blue', linestyle=':', label=f'Sesgo / Bias ({bias:.2f} W)')
plt.title('Análisis de Residuales: Distribución de Errores')
plt.xlabel('Error de Predicción (Real - Predicho) [Vatios]')
plt.ylabel('Frecuencia')
plt.legend()
plt.tight_layout()
plt.savefig('histograma_errores_residuales.png', dpi=300)
plt.show()

print("=== ANÁLISIS ESTADÍSTICO DE RESIDUALES ===")
print(f"Error medio (Sesgo / Bias): {bias:.2f} W")
print(f"Desviación estándar de los residuales: {std_res:.2f} W")
```




```
=== ANÁLISIS ESTADÍSTICO DE RESIDUALES ===
Error medio (Sesgo / Bias): 7.94 W
Desviación estándar de los residuales: 120.36 W
```

CELDA 18: EXPORTACIÓN DE RESULTADOS FINALES Y MODELO ENTRENADO

```
# Guardar modelo Keras
model.save('modelo_red_neuronal_finca.keras')

# Crear DataFrame de salida en test
pred_out_df = pd.DataFrame({
    'timestamp': y_test.index,
    'potencia_real_w': y_true.flatten(),
    'potencia_predicha_w': y_pred.flatten(),
    'error_w': residuales.flatten()
})
pred_out_df.to_csv('predicciones_test_finca.csv', index=False)

# Resumen Ejecutivo Final
print("=== RESUMEN EJECUTIVO Y CONFIRMACIÓN DE ENTREGABLES ===")
print(f"1. Modelo neuronal guardado como 'modelo_red_neuronal_finca.keras'.")
print(f"2. Tabla de predicciones test exportada: 'predicciones_test_finca.csv'.")
print(f"3. Diagnóstico económico guardado en 'diagnostico_finca.csv'.")
print(f"4. Insumos para RAG listos: 'reporte_para_ia_local.md' y 'dataset_faq_ia_local.json'.")
print("\n--- KPIs Claves Generales ---")
print(f"- Consumo Total de Energía: {consumo_real:.2f} kWh/mes")
print(f"- Costo de Factura Estimado: ${costo_total_factura:,.2f} COP")
print(f"- Mitigación CO2 potencial (eliminación Standby): {emisiones_mitigables_kg:.3f} kg CO2e / mes")
print(f"- Eficiencia de Predicción (R² Score): {r2:.4f}")
```

```
=== RESUMEN EJECUTIVO Y CONFIRMACIÓN DE ENTREGABLES ===
1. Modelo neuronal guardado como 'modelo_red_neuronal_finca.keras'.
2. Tabla de predicciones test exportada: 'predicciones_test_finca.csv'.
3. Diagnóstico económico guardado en 'diagnostico_finca.csv'.
4. Insumos para RAG listos: 'reporte_para_ia_local.md' y 'dataset_faq_ia_local.json'.

--- KPIs Claves Generales ---
- Consumo Total de Energía: 297.74 kWh/mes
- Costo de Factura Estimado: $213,545.28 COP
- Mitigación CO2 potencial (eliminación Standby): 2.201 kg CO2e / mes
- Eficiencia de Predicción (R² Score): 0.9394
```

```
# FASE 5: DESPLIEGUE DEL DASHBOARD EJECUTIVO INTERACTIVO (GRADIO 5.0 & PLOTLY)
## CELDA 19: CONSTRUCCIÓN Y LANZAMIENTO DEL DASHBOARD DE NIVEL SAAS PROFESIONAL
```

```
import os
import numpy as np
import pandas as pd
import datetime
import gradio as gr
import plotly.graph_objects as go
import plotly.express as px
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

# ======================================================================
# 1. CARGA DE DATOS & MECANISMO DE RESILIENCIA (FALLBACK SINTÉTICO)
# ======================================================================
PATH_30_DIAS = '/content/mediciones_finca_30dias.csv'
FE_COLOMBIA = 0.126  # Factor de emisión oficial SIN Colombia (kg CO2e / kWh)

try:
    if os.path.exists(PATH_30_DIAS):
        print("Cargando dataset real mediciones_finca_30dias.csv...")
        df = pd.read_csv(PATH_30_DIAS)
        df['timestamp'] = pd.to_datetime(df['timestamp'])
        df = df.set_index('timestamp')
    else:
        raise FileNotFoundError()
except Exception:
    print("Dataset no encontrado. Generando datos sintéticos de alta fidelidad para el Tambo, Cauca..."
    base_date = datetime.datetime(2026, 9, 1, 0, 0)
    timestamps = [base_date + datetime.timedelta(minutes=15 * i) for i in range(2880)]

    # Simulación física realista de parámetros
    voltajes = []
    corrientes = []
    fps = []
    potencias = []
    energias = []
    estados = []

    e_acumulada = 0.0
    for i, t in enumerate(timestamps):
        hour = t.hour
        # Ruido y variaciones típicas de zona rural
        v = 110.0 + 3.0 * np.sin(2 * np.pi * i / 96) + np.random.normal(0, 1.2)

        # Definición de estados y cargas
        if 0 <= hour <= 5:
            estado = "Reposo / Standby (Cámara + Alimentador)"
            p_base = 110.0 + np.random.normal(0, 10.0)
            fp = 0.84 + np.random.normal(0, 0.01)
        elif 19 <= hour <= 21:
            estado = "Pico Noche: Estufa + Iluminación LED + TV + Nevera"
            p_base = 1500.0 + np.random.normal(0, 150.0)
            fp = 0.98 + np.random.normal(0, 0.01)
        elif 12 <= hour <= 14:
            estado = "Almuerzo: Estufa Eléctrica + Nevera"
            p_base = 1200.0 + np.random.normal(0, 100.0)
            fp = 0.97 + np.random.normal(0, 0.01)
        else:
```

```
                estado = "Nevera + Cargadores + TV"
                p_base = 350.0 + np.random.normal(0, 40.0)
                fp = 0.90 + np.random.normal(0, 0.02)

            p_base = max(15.0, p_base)
            fp = max(0.6, min(1.0, fp))
            c = p_base / (v * fp) if v > 0 else 0.0

            # Consumo de energía acumulado (15 min = 0.25 h)
            e_acumulada += (p_base * 0.25 / 1000.0)

            voltajes.append(v)
            corrientes.append(c)
            fps.append(fp)
            potencias.append(p_base)
            energias.append(e_acumulada)
            estados.append(estado)

        df = pd.DataFrame({
            'voltaje_rms_v': voltajes,
            'corriente_rms_a': corrientes,
            'factor_potencia': fps,
            'potencia_activa_w': potencias,
            'energia_acumulada_kwh': energias,
            'estado_estimado': estados
        }, index=pd.to_datetime(timestamps))
        df.index.name = 'timestamp'

    # ======================================================================
    # 2. INGENIERÍA DE CARACTERÍSTICAS Y ENTRENAMIENTO DE IA EN MEMORIA
    # ======================================================================
    print("Entrenando modelo de Inteligencia Artificial (MLPRegressor)... ")
    df_feat = df.copy()
    df_feat['hora'] = df_feat.index.hour
    df_feat['dia_semana'] = df_feat.index.dayofweek
    df_feat['potencia_lag1'] = df_feat['potencia_activa_w'].shift(1)
    df_feat['potencia_lag2'] = df_feat['potencia_activa_w'].shift(2)
    df_feat['potencia_lag4'] = df_feat['potencia_activa_w'].shift(4)
    df_feat = df_feat.dropna()

    # Lista de predictores sin incluir corriente_rms_a para evitar Data Leakage
    PREDICTORES = ['hora', 'dia_semana', 'potencia_lag1', 'potencia_lag2', 'potencia_lag4', 'voltaje_rms_v'
    X_data = df_feat[PREDICTORES]
    y_data = df_feat['potencia_activa_w']

    # Partición cronológica (80% entrenamiento, 20% test)
    split_limit = int(len(X_data) * 0.8)
    X_train, X_test = X_data.iloc[:split_limit], X_data.iloc[split_limit:]
    y_train, y_test = y_data.iloc[:split_limit], y_data.iloc[split_limit:]

    scaler_X = StandardScaler()
    scaler_y = StandardScaler()

    X_train_sc = scaler_X.fit_transform(X_train)
    X_test_sc = scaler_X.transform(X_test)

    y_train_sc = scaler_y.fit_transform(y_train.values.reshape(-1, 1)).ravel()
    y_test_sc = scaler_y.transform(y_test.values.reshape(-1, 1)).ravel()

    # Modelo MLPRegressor super rápido en CPU
    mlp_model = MLPRegressor(hidden_layer_sizes=(32, 16), activation='relu', random_state=42, max_iter=80,
    mlp_model.fit(X_train_sc, y_train_sc)

    # Evaluar Métricas en Test
```

```
y_pred_sc = mlp_model.predict(X_test_sc)
y_pred = scaler_y.inverse_transform(y_pred_sc.reshape(-1, 1)).ravel()
y_true = y_test.values

# Métricas de Precisión globales
mae_val = np.mean(np.abs(y_true - y_pred))
rmse_val = np.sqrt(np.mean((y_true - y_pred) ** 2))
u_y_true = np.mean(y_true)
r2_val = 1.0 - (np.sum((y_true - y_pred) ** 2) / np.sum((y_true - u_y_true) ** 2))

# Guardar diagnóstico estático
diag_df = pd.DataFrame({
    'indicador': ['Consumo Total (kWh)', 'Costo Factura (COP)', 'Emisiones CO2 (kg)', 'R2 Score'],
    'valor': [float(df['potencia_activa_w'].sum() * 0.25 / 1000.0), 0.0, float(df['potencia_activa_w'].
})
diag_df.to_csv('/content/diagnostico_finca.csv', index=False)
print(f"Modelo entrenado exitosamente con R² Score en Test: {r2_val:.4f}")

# ======================================================================
# 3. INTERFAZ GRÁFICA GRADIO (BLOCKS) & DISEÑO TEMA DARK SAAS
# ======================================================================
custom_css = """
body, .gradio-container {
    background-color: #070d19 !important;
    color: #f8fafc !important;
    font-family: 'Inter', system-ui, -apple-system, sans-serif !important;
}
.tabs, .tab-nav {
    border-bottom: 1px solid #1e293b !important;
}
.tab-nav button {
    color: #94a3b8 !important;
    font-weight: 600 !important;
    font-size: 0.95rem !important;
}
.tab-nav button.selected {
    color: #38bdf8 !important;
    border-bottom: 2px solid #38bdf8 !important;
}
.custom-card {
    background-color: #0f172a !important;
    border: 1px solid #1e293b !important;
    border-radius: 12px !important;
    padding: 16px !important;
    margin: 8px 0 !important;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.2) !important;
}
.kpi-title {
    font-size: 0.8rem !important;
    color: #94a3b8 !important;
    text-transform: uppercase !important;
    letter-spacing: 0.05em !important;
    font-weight: bold !important;
}
.kpi-value {
    font-size: 1.6rem !important;
    font-weight: 800 !important;
    color: #ffffff !important;
    margin: 4px 0 !important;
}
.badge-sub {
    background-color: rgba(56, 189, 248, 0.1) !important;
    color: #38bdf8 !important;
    padding: 3px 8px !important;
```

```
        border-radius: 9999px !important;
        font-size: 0.72rem !important;
        font-weight: bold !important;
        display: inline-block !important;
    }
    .badge-danger {
        background-color: rgba(239, 68, 68, 0.1) !important;
        color: #ef4444 !important;
        padding: 3px 8px !important;
        border-radius: 9999px !important;
        font-size: 0.72rem !important;
        font-weight: bold !important;
        display: inline-block !important;
    }
    """

    def callback_actualizar(filtro_estados, tarifa_sub, tarifa_plena, lim_sub, cargo_fijo, hsp, panel_wp):
        # Filtrar copia local del dataframe
        d_local = df.copy()
        if filtro_estados:
            d_local = d_local[d_local['estado_estimado'].isin(filtro_estados)]

        if d_local.empty:
            return ["<p style='color:red;'>Seleccione al menos una carga para calcular.</p>"] * 4 + [go.Fig

        # 1. Cálculos Eléctricos y CREG
        consumo_kwh_mes = float(d_local['potencia_activa_w'].sum() * 0.25 / 1000.0)
        tramo_sub_calc = min(consumo_kwh_mes, lim_sub)
        tramo_exc_calc = max(0.0, consumo_kwh_mes - lim_sub)

        costo_sub_cop = tramo_sub_calc * tarifa_sub
        costo_exc_cop = tramo_exc_calc * tarifa_plena
        factura_total_cop = costo_sub_cop + costo_exc_cop + cargo_fijo

        # 2. Análisis Standby
        madrugada_local = d_local.between_time('00:00', '05:00')
        avg_standby_w = float(madrugada_local['potencia_activa_w'].mean()) if not madrugada_local.empty els
        kwh_standby_mes = float(madrugada_local['potencia_activa_w'].sum() * 0.25 / 1000.0) if not madrugad
        costo_promedio_kwh = (costo_sub_cop + costo_exc_cop) / consumo_kwh_mes if consumo_kwh_mes > 0 else
        costo_standby_cop = kwh_standby_mes * costo_promedio_kwh

        # 3. Emisiones
        co2_total_kg = consumo_kwh_mes * FE_COLOMBIA

        # 4. Formatear tarjetas en gr.HTML
        badge_proy = f"<span class='badge-sub'>Optimizado</span>" if consumo_kwh_mes <= lim_sub else f"<spa

        kpi1 = f"<div class='custom-card'><div class='kpi-title'>Consumo Total</div><div class='kpi-value'>
        kpi2 = f"<div class='custom-card'><div class='kpi-title'>Factura Estimada</div><div class='kpi-valu
        kpi3 = f"<div class='custom-card'><div class='kpi-title'>Fuga Standby (00-05h)</div><div class='kpi
        kpi4 = f"<div class='custom-card'><div class='kpi-title'>Huella CO2 (ISO 14064)</div><div class='kp

        # --- PESTAÑA 1: PREDICCIÓN & RESIDUALES (PLOTLY) ---
        horizonte_test = y_test.index[-96:]  # Últimas 24 horas del conjunto de test
        y_reales_plot = y_true[-96:]
        y_predichos_plot = y_pred[-96:]

        fig_pred = go.Figure()
        fig_pred.add_trace(go.Scatter(x=horizonte_test, y=y_reales_plot, name='Real Medido', line=dict(colo
        fig_pred.add_trace(go.Scatter(x=horizonte_test, y=y_predichos_plot, name='Pronóstico IA (MLP)', lin
        fig_pred.update_layout(
            template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
            title="Horizonte de Predicción 24 Horas con Red Neuronal", xaxis_title="Timestamp", yaxis_title
        )
```

```
errores_test = y_true - y_pred
fig_hist_err = px.histogram(errores_test, nbins=40, color_discrete_sequence=['#ef4444'], title="Dis
fig_hist_err.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgb

# --- PESTAÑA 2: PERFILES DE CARGA & SEUs ---
fig_timeline = go.Figure()
fig_timeline.add_trace(go.Scatter(x=d_local.index, y=d_local['potencia_activa_w'], name='Consumo (W
fig_timeline.add_hline(y=1000.0, line=dict(color='#ef4444', dash='dash'), annotation_text="Umbral T
fig_timeline.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgb

# Pareto SEUs
pareto_data = d_local.groupby('estado_estimado')['potencia_activa_w'].sum().reset_index().sort_valu
total_pot = pareto_data['potencia_activa_w'].sum()
pareto_data['porcentaje_acumulado'] = (pareto_data['potencia_activa_w'] / total_pot * 100).cumsum()

fig_pareto = go.Figure()
fig_pareto.add_bar(x=pareto_data['estado_estimado'], y=pareto_data['potencia_activa_w'], name='Cons
fig_pareto.add_trace(go.Scatter(x=pareto_data['estado_estimado'], y=pareto_data['porcentaje_acumula
fig_pareto.update_layout(
    template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
    title="Análisis de Pareto 80/20 de Sistemas Significativos (SEUs)",
    yaxis=dict(title="Potencia Sumada (W)"),
    yaxis2=dict(title="% Acumulado", overlaying='y', side='right', range=[0, 105]),
    xaxis=dict(tickangle=-30)
)

# --- PESTAÑA 3: TRANSICIÓN SOLAR CREG 174 ---
efectividad_solar = 0.75  # Performance Ratio de instalación estándar
gen_panel_anual = (panel_wp * hsp * efectividad_solar * 365.0) / 1000.0  # kWh al año por módulo

excedente_anual_kwh = tramo_exc_calc * 12.0
if gen_panel_anual > 0 and excedente_anual_kwh > 0:
    paneles_necesarios = int(np.ceil(excedente_anual_kwh / gen_panel_anual))
else:
    paneles_necesarios = 0

area_m2 = paneles_necesarios * 2.1  # Promedio m² por panel comercial de 400Wp
ahorro_anual_cop = excedente_anual_kwh * tarifa_plena
costo_inversion_aprox_cop = paneles_necesarios * 1800000.0  # Costo rural instalado COP

payback_years = (costo_inversion_aprox_cop / ahorro_anual_cop) if ahorro_anual_cop > 0 else 0.0

# KPI Box fotovoltaico
solar_kpis_html = f"""
<div style='display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 12px;'
    <div class='custom-card'><div class='kpi-title'>Módulos Necesarios (400Wp)</div><div class='kpi
    <div class='custom-card'><div class='kpi-title'>Área Techo Estimada</div><div class='kpi-value'
    <div class='custom-card'><div class='kpi-title'>Ahorro Anual Proyectado</div><div class='kpi-va
    <div class='custom-card'><div class='kpi-title'>Retorno de Inversión</div><div class='kpi-value
</div>
"""

# --- PESTAÑA 4: MATRIZ DE CORRELACIÓN & FINANZAS ---
heat_cols = ['voltaje_rms_v', 'corriente_rms_a', 'factor_potencia', 'potencia_activa_w']
heat_corr = d_local[heat_cols].corr(method='pearson')
fig_heat = px.imshow(heat_corr, text_auto='.3f', color_continuous_scale='RdBu_r', title="Matriz de
fig_heat.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,

fig_dona = go.Figure(data=[go.Pie(
    labels=['Costo Subsidiado', 'Costo Excedente Pleno', 'Cargo Fijo'],
    values=[costo_sub_cop, costo_exc_cop, cargo_fijo],
    hole=0.45,
    marker=dict(colors=['#38bdf8', '#ef4444', '#10b981'])
```

```
        )])
        fig_dona.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,

        # --- PESTAÑA 5: CONCLUSIONES RAG EN MARKDOWN ---
        reporte_rag_md = f"""
        ## 📋 DIAGNÓSTICO METROLÓGICO E INFORME DE CAUSA RAÍZ (ISO 50001)

        * **Causa Raíz de Sobrecosto:** El consumo actual de la finca (**{consumo_kwh_mes:.2f} kWh/mes**) e

        ### 🔋 Plan Inmediato de Eficiencia de 3 Medidas:
        1. **Eliminar Consumo Fantasma (Costo $0 COP):** Interrupción automatizada o desconexión manual de
        2. **Mitigación del Pico Nocturno:** Desplazamiento de las cargas de cocción del tramo crítico (19:
        3. **Autogeneración Solar Fotovoltaica (CREG 174):** Implementar el sistema dimensionado de **{pane
        """

        return (
            kpi1, kpi2, kpi3, kpi4,
            fig_pred, fig_hist_err,
            fig_timeline, fig_pareto,
            solar_kpis_html,
            fig_heat, fig_dona,
            reporte_rag_md
        )

    # ======================================================================
    # 4. DEFINICIÓN DEL LAYOUT DE GRADIO INTERACTIVO
    # ======================================================================
    lista_cargas = sorted(df['estado_estimado'].unique().tolist())

    with gr.Blocks(css=custom_css, title="Dashboard Energético Inteligente SaaS 5.0") as demo:
        gr.HTML("""
        <div style='text-align: center; margin-bottom: 20px; border-bottom: 2px solid #1e293b; padding-bott
            <h1 style='color: #38bdf8; font-size: 2.2rem; margin: 0; font-weight: 800;'>⚡ SaaS Centro de C
            <p style='color: #94a3b8; font-size: 1rem; margin-top: 5px;'>Monitoreo Industrial, Modelado con
        </div>
        """)

        # Tarjetas Bento superiores (Layout Fijo de KPIs)
        with gr.Row():
            kpi_html_1 = gr.HTML()
            kpi_html_2 = gr.HTML()
            kpi_html_3 = gr.HTML()
            kpi_html_4 = gr.HTML()

        with gr.Row():
            # Panel Lateral de Control
            with gr.Column(scale=1, min_width=320):
                gr.HTML("<div style='border-bottom: 1px solid #1e293b; margin-bottom: 10px;'><h3 style='col

                cb_filtros = gr.CheckboxGroup(
                    choices=lista_cargas,
                    value=lista_cargas,
                    label="Filtrar por Cargas Activas (SEUs)"
                )

                with gr.Accordion("💰 Parámetros Tarifarios CEO/CREG", open=False):
                    val_tar_sub = gr.Slider(minimum=300, maximum=800, value=480, step=10, label="Tarifa Sub
                    val_tar_plena = gr.Slider(minimum=700, maximum=1500, value=950, step=10, label="Tarifa
                    val_lim_sub = gr.Slider(minimum=100, maximum=250, value=173, step=1, label="Límite Subs
                    val_cargo_fijo = gr.Slider(minimum=5000, maximum=30000, value=12000, step=500, label="C

                with gr.Accordion("☀️ Configuración de Paneles Solares", open=False):
                    val_hsp = gr.Slider(minimum=2.5, maximum=6.0, value=4.2, step=0.1, label="Horas Sol Pic
                    val_panel_wp = gr.Slider(minimum=300, maximum=550, value=400, step=10, label="Potencia
```

```
            btn_recalc = gr.Button("🔄 RECALCULAR DASHBOARD", variant="primary")

        # Sección Central de Pestañas Técnicas
        with gr.Column(scale=3):
            with gr.Tabs():
                with gr.TabItem("🧠 Predicción & IA"):
                    gr.Markdown(f"""
                    ### Red Neuronal de Pronóstico Multicapa (MLPRegressor)
                    * **Métricas Globales Evaluadas:**
                      * **Coeficiente de Determinación (R² Score):** {r2_val:.4f}
                      * **Error Absoluto Medio (MAE):** {mae_val:.2f} W
                      * **Raíz del Error Cuadrático Medio (RMSE):** {rmse_val:.2f} W
                    """)
                    with gr.Row():
                        p1_plot_pred = gr.Plot(label="Predicción 24h")
                        p1_plot_hist = gr.Plot(label="Distribución de Error")

                with gr.TabItem("📊 Perfiles de Carga & SEUs"):
                    with gr.Row():
                        p2_plot_time = gr.Plot(label="Línea Temporal de Demanda")
                    with gr.Row():
                        p2_plot_pareto = gr.Plot(label="Gráfico de Pareto")

                with gr.TabItem("☀️ Transición Solar CREG 174"):
                    gr.Markdown("### Simulación Automatizada para Autogenerador a Pequeña Escala (AGPE)
                    p3_solar_kpis = gr.HTML()
                    gr.Markdown("""
                    * **Ley 1715 & CREG 174:** Permite a los usuarios residenciales rurales vender ener
                    * **Área Útil de Techo:** Calculada estimando 2.1 m² por módulo de 400Wp, considera
                    """)

                with gr.TabItem("💰 Tarifas & Correlación"):
                    with gr.Row():
                        p4_plot_heat = gr.Plot(label="Matriz de Correlación")
                        p4_plot_dona = gr.Plot(label="Distribución Recibo Eléctrico")

                with gr.TabItem("📋 Conclusiones RAG"):
                    p5_md_rag = gr.Markdown()

    # Evento para orquestar la actualización dinámica de toda la app
    entradas_control = [
        cb_filtros, val_tar_sub, val_tar_plena, val_lim_sub, val_cargo_fijo, val_hsp, val_panel_wp
    ]

    salidas_dashboard = [
        kpi_html_1, kpi_html_2, kpi_html_3, kpi_html_4,
        p1_plot_pred, p1_plot_hist,
        p2_plot_time, p2_plot_pareto,
        p3_solar_kpis,
        p4_plot_heat, p4_plot_dona,
        p5_md_rag
    ]

    # Vinculación de eventos de carga y botón de actualizar
    btn_recalc.click(fn=callback_actualizar, inputs=entradas_control, outputs=salidas_dashboard)
    demo.load(fn=callback_actualizar, inputs=entradas_control, outputs=salidas_dashboard)

# Lanzamiento del dashboard con enlace público en Google Colab
demo.launch(share=True, debug=False)
```

```
Cargando dataset real mediciones_finca_30dias.csv...
Entrenando modelo de Inteligencia Artificial (MLPRegressor)...
/usr/local/lib/python3.13/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: Convergenc
  warnings.warn(
/tmp/ipykernel_3654/2950590937.py:343: UserWarning: The parameters have been moved from the Blocks const
  with gr.Blocks(css=custom_css, title="Dashboard Energético Inteligente SaaS 5.0") as demo:
Modelo entrenado exitosamente con R² Score en Test: 0.9309
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://5adbb0a245b963ae5c.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting an
```

SaaS Centro de Control Energético Rural

Monitoreo Industrial, Modelado con Inteligencia Artificial & Transición Fotovoltaica | El Tambo, Cauca

CONSUMO TOTAL	FACTURA ESTIMADA	FUGA STANDBY (00-05H)	HUELLA CO2 (ISO 14064)
297.74 kWh	$ 213,545.28	110.9 W	37.52 kg
+72.1% Exceso	$ 83,040 Sub. | $ 118,505 Exc.	$ 11,826.75 COP/mes	1.25 kg CO2e/día

🔧 Parámetros de Simulación 🧠 Predicción & IA •••

Próximos pasos: Implementar en Cloud Run

```python
import gradio as gr
import plotly.graph_objects as go
import plotly.express as px

# --- 1. CONFIGURACIÓN DEL INJECTED CSS (Tailwind Dark & SaaS Theme) ---
custom_css = """
body, .gradio-container {
    background-color: #070d19 !important;
    color: #f8fafc !important;
    font-family: 'Inter', system-ui, -apple-system, sans-serif !important;
}
.tabs, .tab-nav {
    border-bottom: 1px solid #1e293b !important;
}
.tab-nav button {
    color: #94a3b8 !important;
    font-weight: 600 !important;
}
.tab-nav button.selected {
    color: #38bdf8 !important;
    border-bottom: 2px solid #38bdf8 !important;
}
.custom-card {
```

```
.custom-card {
    background-color: #0f172a !important;
    border: 1px solid #1e293b !important;
    border-radius: 16px !important;
    padding: 18px !important;
    margin-bottom: 15px !important;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06) !important;
}
.kpi-title {
    font-size: 0.85rem !important;
    color: #94a3b8 !important;
    text-transform: uppercase !important;
    letter-spacing: 0.05em !important;
}
.kpi-value {
    font-size: 1.8rem !important;
    font-weight: 700 !important;
    color: #ffffff !important;
    margin: 5px 0 !important;
}
.badge-red {
    background-color: rgba(239, 68, 68, 0.15) !important;
    color: #ef4444 !important;
    padding: 4px 8px !important;
    border-radius: 9999px !important;
    font-size: 0.75rem !important;
    font-weight: 600 !important;
}
.badge-orange {
    background-color: rgba(245, 158, 11, 0.15) !important;
    color: #f59e0b !important;
    padding: 4px 8px !important;
    border-radius: 9999px !important;
    font-size: 0.75rem !important;
    font-weight: 600 !important;
}
.badge-green {
    background-color: rgba(16, 185, 129, 0.15) !important;
    color: #10b981 !important;
    padding: 4px 8px !important;
    border-radius: 9999px !important;
    font-size: 0.75rem !important;
    font-weight: 600 !important;
}
"""

# --- 2. MOTOR DE CÁLCULO E INTERACTIVIDAD DEL DASHBOARD ---
def actualizar_analisis(estados_filtrar, tarifa_sub, tarifa_plena, lim_sub, cargo_fijo, hsp, panel_wp, pr
    # Copia de seguridad local para no alterar el estado global de ejecución
    d_temp = df.copy()

    # Filtrar por estados estimados seleccionados
    if estados_filtrar:
        d_temp = d_temp[d_temp['estado_estimado'].isin(estados_filtrar)]

    if d_temp.empty:
        return ["<p style='color:red;'>Por favor seleccione al menos una carga.</p>"] * 10

    # Recalcular Consumo total del mes (kWh)
    con_mensual = d_temp['potencia_activa_w'].sum() * 0.25 / 1000

    # Liquidación tarifaria modificada
    tramo_s = min(con_mensual, lim_sub)
    tramo_e = max(0.0, con_mensual - lim_sub)
```

```
    cost_sub = tramo_s * tarifa_sub
    cost_exc = tramo_e * tarifa_plena
    cost_total = cost_sub + cost_exc + cargo_fijo

    # Alerta porcentual sobre el límite de subsistencia
    porc_desviacion = ((con_mensual - lim_sub) / lim_sub) * 100
    if porc_desviacion > 0:
        badge_proyeccion = f"<span class='badge-red'>+{porc_desviacion:.1f}% sobre límite de {lim_sub:.0f
    else:
        badge_proyeccion = f"<span class='badge-green'>Bajo el límite de subsistencia rural</span>"

    # Fuga de Standby Recalculada para el subgrupo de madrugada
    mad_temp = d_temp.between_time('00:00', '05:00')
    standby_w_act = mad_temp['potencia_activa_w'].mean() if not mad_temp.empty else 0
    standby_kwh_act = (mad_temp['potencia_activa_w'] * 0.25 / 1000).sum() if not mad_temp.empty else 0
    costo_kwh_medio = (cost_sub + cost_exc) / con_mensual if con_mensual > 0 else 0
    perdida_standby_cop = standby_kwh_act * costo_kwh_medio

    # Huella de Carbono
    co2_act = con_mensual * FE_COLOMBIA

    # --- GENERAR HTML DE LAS TARJETAS KPIS ---
    kpi1_html = f"""
    <div class='custom-card'>
        <div class='kpi-title'>Consumo Proyectado</div>
        <div class='kpi-value'>{con_mensual:.2f} kWh/mes</div>
        {badge_proyeccion}
    </div>
    """

    kpi2_html = f"""
    <div class='custom-card'>
        <div class='kpi-title'>Costo Recibo Mensual</div>
        <div class='kpi-value'>$ {cost_total:,.2f} COP</div>
        <span class='badge-orange'>$ {cost_sub:,.0f} Sub. + $ {cost_exc:,.0f} Exc.</span>
    </div>
    """

    kpi3_html = f"""
    <div class='custom-card'>
        <div class='kpi-title'>Standby en Madrugada</div>
        <div class='kpi-value'>{standby_w_act:.1f} W prom.</div>
        <span class='badge-red'>$ {perdida_standby_cop:,.0f} COP/mes en pérdidas</span>
    </div>
    """

    kpi4_html = f"""
    <div class='custom-card'>
        <div class='kpi-title'>Huella de Carbono</div>
        <div class='kpi-value'>{co2_act:.2f} kg CO2e</div>
        <span class='badge-green'>ISO 14064-1 · {co2_act/30:.2f} kg/día</span>
    </div>
    """

    # --- FIGURA 1: CALIDAD DE VOLTAJE (PLOTLY) ---
    fig_voltaje = go.Figure()
    fig_voltaje.add_trace(go.Scatter(
        x=d_temp.index, y=d_temp['voltaje_rms_v'],
        name='Voltaje Real (V)', line=dict(color='#38bdf8', width=1.5)
    ))
    fig_voltaje.add_hline(y=110, line=dict(color='#ffffff', dash='solid'), annotation_text="110 V Nominal
    fig_voltaje.add_hline(y=121, line=dict(color='#ef4444', dash='dash'), annotation_text="+10% Toleranci
    fig_voltaje.add_hline(y=99, line=dict(color='#ef4444', dash='dash'), annotation_text="-10% Tolerancia
    fig_voltaje.update_layout(
        template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
```

```
        title="Calidad de Tensión Rural vs Estándares RETIE/CREG",
        xaxis_title="Línea de Tiempo", yaxis_title="Voltaje RMS (V)"
    )

    # --- FIGURA 2: CURVA TEMPORAL DE DEMANDA ---
    fig_potencia = go.Figure()
    fig_potencia.add_trace(go.Scatter(
        x=d_temp.index, y=d_temp['potencia_activa_w'],
        name='Demanda Activa (W)', fill='tozeroy',
        fillcolor='rgba(56, 189, 248, 0.1)', line=dict(color='#38bdf8', shape='spline')
    ))
    fig_potencia.add_hline(y=1000, line=dict(color='#ef4444', dash='dot', width=2), annotation_text="Umbr
    fig_potencia.update_layout(
        template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
        title="Serie Temporal de Demanda de Potencia Activa (30 días)",
        xaxis_title="Fecha", yaxis_title="Potencia Activa (W)"
    )

    # --- FIGURA 3: PERFIL HORARIO PROMEDIO ---
    hourly_metrics = d_temp.groupby(d_temp.index.hour)['potencia_activa_w'].agg(['mean', 'std'])
    fig_horario = go.Figure()
    fig_horario.add_trace(go.Scatter(
        x=hourly_metrics.index, y=hourly_metrics['mean'],
        name='Demanda Horaria Promedio',
        line=dict(color='#10b981', width=3, shape='spline')
    ))
    fig_horario.add_trace(go.Scatter(
        x=hourly_metrics.index, y=hourly_metrics['mean'] + hourly_metrics['std'],
        mode='lines', line=dict(width=0), showlegend=False
    ))
    fig_horario.add_trace(go.Scatter(
        x=hourly_metrics.index, y=hourly_metrics['mean'] - hourly_metrics['std'],
        mode='lines', line=dict(width=0), fill='tonexty',
        fillcolor='rgba(16, 185, 129, 0.1)', name='Desviación Estándar (±1 std)'
    ))
    fig_horario.update_layout(
        template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
        title="Curva Diaria Promedio de Demanda Electrónica (Ciclo de 24h)",
        xaxis=dict(tickmode='array', tickvals=list(range(24))), xaxis_title="Hora del Día", yaxis_title="
    )

    # --- FIGURA 4: PARETO DE SEUs ---
    seus_df = d_temp.groupby('estado_estimado')['potencia_activa_w'].mean().reset_index().sort_values(by=
    total_seus_pot = seus_df['potencia_activa_w'].sum()
    seus_df['porcentaje_acumulado'] = (seus_df['potencia_activa_w'] / total_seus_pot * 100).cumsum()

    fig_pareto = go.Figure()
    fig_pareto.add_trace(go.Bar(
        x=seus_df['estado_estimado'], y=seus_df['potencia_activa_w'],
        name='Potencia Activa Promedio (W)', marker_color='#38bdf8'
    ))
    fig_pareto.add_trace(go.Scatter(
        x=seus_df['estado_estimado'], y=seus_df['porcentaje_acumulado'],
        name='% Acumulado', yaxis='y2', line=dict(color='#f59e0b', width=2, marker=dict(symbol='diamond')
    ))
    fig_pareto.update_layout(
        template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
        title="Diagrama de Pareto de Sistemas Significativos de Consumo (SEUs)",
        yaxis=dict(title="Potencia Activa Promedio (W)"),
        yaxis2=dict(title="Porcentaje Acumulado (%)", overlaying='y', side='right', range=[0, 105]),
        showlegend=True, legend=dict(x=0.01, y=0.99)
    )

    # --- FIGURA 5: DESGLOSE DE FACTURA (DONA FINANCIERA) ---
```

```
fig_dona_fin = go.Figure(data=[go.Pie(
    labels=['Subsidiado (CREG)', 'Excedente Pleno', 'Cargos Fijos y Red'],
    values=[cost_sub, cost_exc, cargo_fijo],
    hole=.45,
    marker=dict(colors=['#38bdf8', '#ef4444', '#10b981'])
)])
fig_dona_fin.update_layout(
    template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
    title="Desglose Financiero Integrado de la Factura"
)

# --- FIGURA 6: MATRIZ DE CORRELACIÓN DE PEARSON INTERACTIVA ---
p_cols = ['voltaje_rms_v', 'corriente_rms_a', 'factor_potencia', 'potencia_activa_w']
p_corr = d_temp[p_cols].corr(method='pearson')
fig_heatmap = px.imshow(
    p_corr, text_auto='.3f', color_continuous_scale='RdBu_r',
    title="Validación Metrológica: Matriz de Correlación de Pearson",
    aspect="auto"
)
fig_heatmap.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0

# --- MODELO DE DEEP LEARNING (PREDICCIÓN PRÓXIMAS 24 HORAS) ---
# Simulamos el horizonte de predicción de las últimas 96 muestras (24 horas) en test
horizonte_test_index = y_test.index[-96:]
val_reales = y_true[-96:].flatten()
val_predichos = y_pred[-96:].flatten()

fig_predicciones = go.Figure()
fig_predicciones.add_trace(go.Scatter(
    x=horizonte_test_index, y=val_reales,
    name='Demanda Eléctrica Real', line=dict(color='#06b6d4', width=2)
))
fig_predicciones.add_trace(go.Scatter(
    x=horizonte_test_index, y=val_predichos,
    name='Pronóstico Red Neuronal (MLP)', line=dict(color='#ec4899', width=2, dash='dash')
))
fig_predicciones.update_layout(
    template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
    title="Predicción Dinámica con IA Frente a Consumo Real (Últimas 24 Horas de Test)",
    xaxis_title="Línea de Tiempo", yaxis_title="Potencia Activa (W)"
)

# --- FIGURA 8: HISTOGRAMA DE RESIDUALES ---
errores = val_reales - val_predichos
fig_residuales = px.histogram(
    errores, nbins=30, color_discrete_sequence=['#ef4444'],
    title="Distribución de Errores de Predicción de la Inteligencia Artificial"
)
fig_residuales.update_layout(
    template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
    xaxis_title="Error de Predicción (Real - Predicho) [W]", yaxis_title="Frecuencia"
)

# --- TRANSICIÓN SOLAR FOTOVOLTAICA (CREG 174 & LEY 1715) ---
generacion_anual_esperada = (panel_wp * hsp * pr * 365) / 1000 # kWh/año por módulo
excedente_anual_a_cubrir = tramo_e * 12 # kWh/año acumulados

paneles_necesarios = max(1.0, np.ceil(excedente_anual_a_cubrir / generacion_anual_esperada)) if gener
capacidad_instalacion_kw = (paneles_necesarios * panel_wp) / 1000
area_util = paneles_necesarios * 2.1 # aprox. 2.1m2 por panel

solar_kpis_html = f"""
<div style='display: grid; grid-template-columns: repeat(4, 1fr); gap: 15px;'>
    <div class='custom-card'>
        <div class='kpi-title'>Energía Excedente a Cubrir</div>
```